

CreditCompass: An Intelligent Credit Risk Scoring System Using XGBoost

Team Async
Newton School of Technology

ABSTRACT

CreditCompass is a production-ready credit risk assessment system built using XGBoost. The system predicts the probability of borrower default using financial and behavioral indicators. It provides real-time predictions through a Flask-based REST API and includes a responsive web dashboard for interaction. The project focuses not only on predictive performance but also on deployment readiness, scalability, and practical usability in financial environments. Experimental results show that XGBoost outperforms traditional models in accuracy and overall classification performance.

INDEX TERMS

Credit risk, XGBoost, Machine learning, Flask, REST API, Financial analytics.

I. INTRODUCTION

Credit risk assessment plays a crucial role in financial lending systems. Accurately identifying high-risk borrowers helps institutions reduce losses and improve decision-making.

CreditCompass is designed as an end-to-end intelligent credit scoring system. It combines machine learning with a deployable web application so that predictions can be generated in real time. The system integrates model training, API development, validation, and cloud deployment within a unified workflow.

The primary objective of this project is to build a reliable and deployable credit risk scoring solution suitable for real-world applications.

II. RELATED WORK

Traditional credit scoring approaches commonly rely on Logistic Regression due to its interpretability. Ensemble methods such as Random Forest and Gradient Boosting have shown improved predictive performance for structured financial data.

XGBoost has become widely used in financial risk modeling because it provides strong accuracy, regularization support, and efficient handling of large datasets.

III. METHODOLOGY

A. Dataset Description

The dataset consists of borrower financial attributes, including:

- Revolving credit utilization ratio
- Debt-to-income ratio
- Monthly income

- Number of open credit lines
- Late payment counts (30–59, 60–89, and 90+ days)
- Number of real estate loans
- Number of dependents

The target variable represents whether the borrower is likely to default.

B. Preprocessing

Data preprocessing steps include:

- Handling missing values
- Validating input ranges
- Removing extreme outliers
- Preparing features for model training

C. Model Selection

XGBoost was selected as the primary model because of its strong performance on tabular datasets and built-in regularization techniques that help reduce overfitting.

D. Training and Evaluation

The dataset was divided into training and testing sets using stratified sampling. Model performance was evaluated using:

- Accuracy
- Precision
- Recall
- F1-score

IV. RESULTS

A. Model Performance

TABLE I
MODEL PERFORMANCE COMPARISON

Model	Accuracy	Precision	Recall	F1
Logistic Regression	0.78	0.74	0.70	0.72
Random Forest	0.83	0.80	0.78	0.79
XGBoost	0.86	0.84	0.81	0.82

XGBoost achieved the highest overall performance across all evaluation metrics.

B. Risk Classification

V. SYSTEM ARCHITECTURE

The system consists of:

- A Flask-based REST API backend
- A serialized XGBoost model
- A responsive frontend dashboard
- Deployment configurations for cloud platforms

TABLE II
RISK LEVEL CLASSIFICATION

Probability Range	Risk Level	Interpretation
0–40%	Low Risk	Low probability of default
40–70%	Medium Risk	Moderate probability of default
70–100%	High Risk	High probability of default

A. Prediction Endpoint

```
@app.route("/predict", methods=["POST"])
def predict():
    data = request.get_json()
    probability = model.predict_proba([
        features])[0][1]
    return jsonify({
        "risk_level": risk_level,
        "probability": probability
    })
```

Listing 1. POST /predict

B. Health Check Endpoint

```
@app.route("/health")
def health():
    return jsonify({
        "status": "healthy",
        "model_loaded": True
    })
```

Listing 2. GET /health

VI. DEPLOYMENT

CreditCompass supports deployment on platforms such as Render, Vercel, Railway, AWS Elastic Beanstalk, and VPS environments using Gunicorn and Nginx.

The system includes:

- CORS configuration
- Environment variable management
- Production WSGI server setup
- Reverse proxy configuration

VII. SECURITY CONSIDERATIONS

Security measures include:

- Server-side input validation
- Strict type and range checking
- No storage of sensitive borrower data
- Error handling to prevent information exposure

VIII. CONCLUSION

CreditCompass demonstrates how a machine learning model can be transformed into a deployable and practical financial tool. By integrating XGBoost with a REST API and cloud-ready deployment configurations, the system provides a scalable solution for credit risk assessment.

Future improvements may include model explainability techniques, automated retraining pipelines, and user authentication features.

REFERENCES

- [1] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” KDD, 2016.
- [2] L. Breiman, “Random Forests,” Machine Learning, 2001.
- [3] D. W. Hosmer and S. Lemeshow, Applied Logistic Regression, Wiley.

APPENDIX

The complete source code, trained model, and deployment configuration are available at:

<https://github.com/shibadityadeb/CreditCompass>